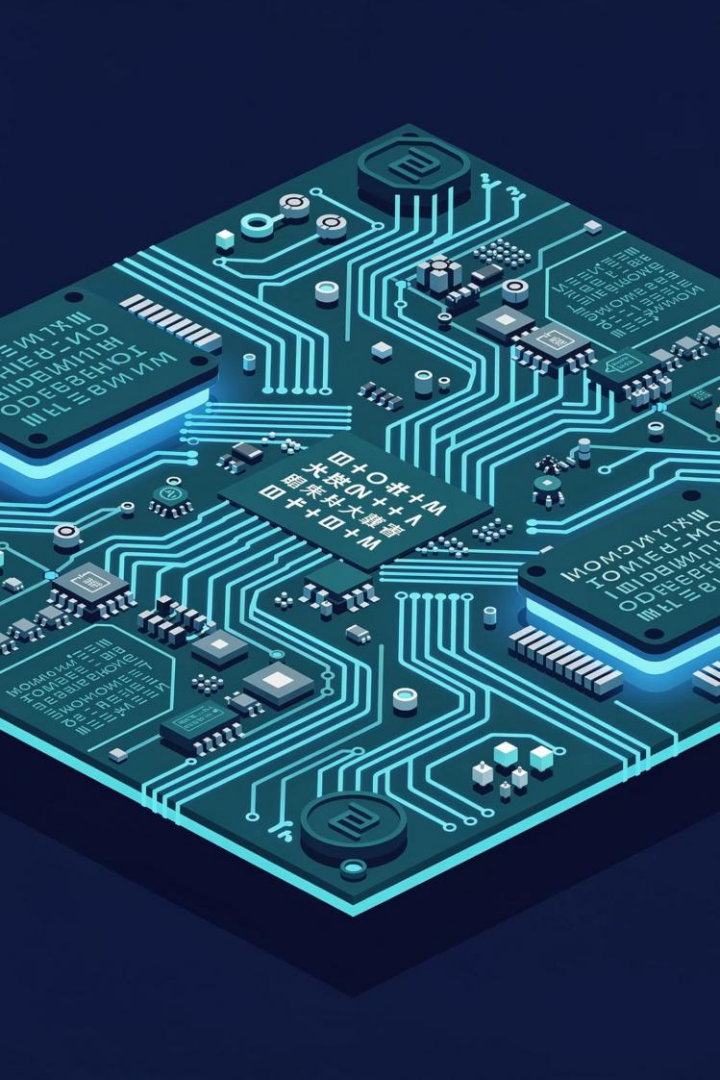




Testing Automatizado y Seguridad en el Software

Introducción a la Programación 2 · Unidad 4

Herramientas, flujos y buenas prácticas para desarrollar software confiable y seguro en el mundo real.

Agenda: Mapa de Contenidos

Esta sesión se organiza en dos grandes bloques temáticos. Al finalizar, serás capaz de diseñar pruebas automatizadas y aplicar principios de seguridad en tus proyectos de software.

Bloque 1: Testing Automatizado

- ¿Qué es la automatización de pruebas?
- Pruebas manuales vs. automatizadas
- Flujo básico y herramientas populares
- Validación funcional y ejemplo real
- IA aplicada al QA

Bloque 2: Seguridad en el Software

- ¿Por qué importa la seguridad?
- Seguridad en el SDLC y Shift Left
- Buenas prácticas de desarrollo seguro
- Seguridad Web y Móvil
- OWASP Top 10

¿Qué es la Automatización de Pruebas?

La **automatización de pruebas** consiste en el uso de herramientas y scripts para ejecutar casos de prueba de forma programática, sin intervención humana constante. A diferencia de las pruebas manuales —donde un tester ejecuta cada paso manualmente— los scripts automatizados replican ese proceso de manera rápida, precisa y repetible.

Imagina a un robot de ensamblaje industrial: realiza la misma tarea miles de veces sin fatigarse ni cometer errores por descuido. Los scripts de prueba funcionan exactamente igual: verifican el comportamiento del software con consistencia absoluta en cada ejecución.



Automatizado

Scripts ejecutan casos de prueba. Velocidad alta, sin fatiga.



Manual

Tester ejecuta cada paso. Flexible pero lento y costoso.

Pruebas Manuales vs. Automatizadas

Cada enfoque tiene su lugar en el ciclo de desarrollo. La clave es saber cuándo usar cada uno.

Criterio	🧑 Pruebas Manuales	🤖 Pruebas Automatizadas
Velocidad	Lenta — depende del tester	Alta — ejecución en segundos
Costo inicial	Bajo	Alto (inversión en scripts)
Repetibilidad	Variable — errores humanos	Perfecta — mismo resultado siempre
Cobertura	Limitada por tiempo y recursos	Alta — miles de casos posibles
Mejor uso	Pruebas exploratorias, UX	Regresión, integración continua

📌 La automatización no reemplaza las pruebas manuales — las complementa. El objetivo es liberar al tester para tareas creativas y exploratorias.

Beneficios de la Automatización



Velocidad

Ejecuta miles de pruebas en minutos, acelerando los ciclos de entrega.



Mayor Cobertura

Cubre escenarios que serían imposibles de testear manualmente por tiempo o complejidad.



Repetibilidad

Los mismos tests se ejecutan de forma idéntica en cada build, sin variación.



Ahorro de Costos

Reducción significativa de costos a largo plazo al minimizar regresiones en producción.

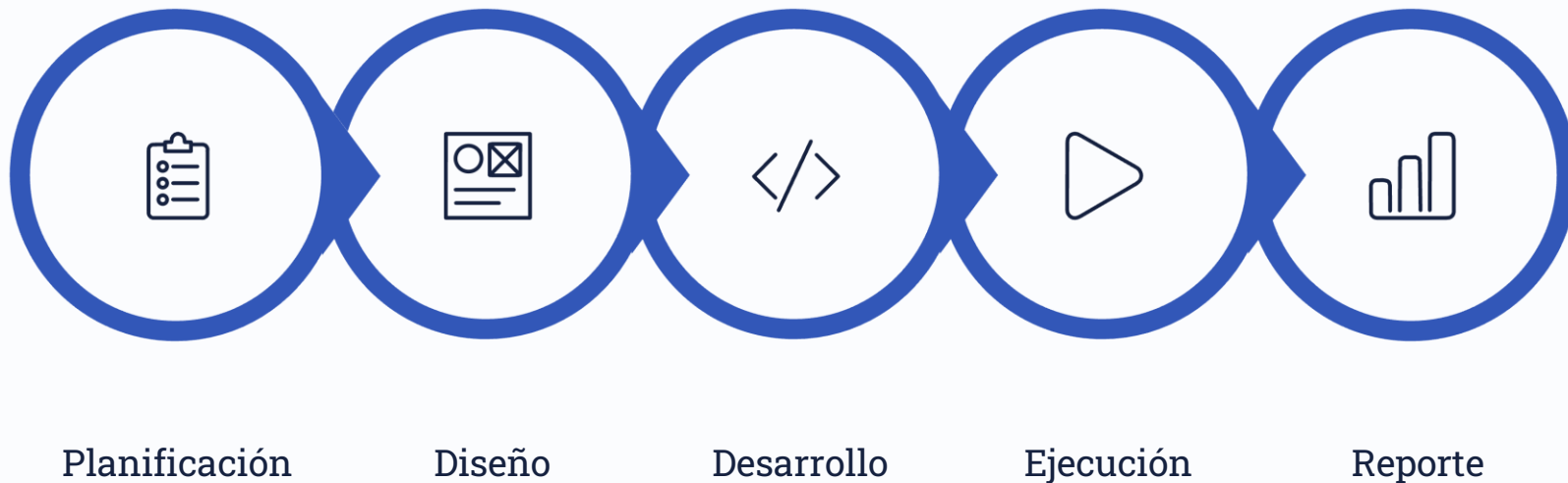


Detección Temprana

Los errores se detectan en desarrollo, cuando corregirlos es hasta 10 veces más barato.

Flujo Básico de Pruebas Automatizadas

Un proceso de automatización efectivo no comienza escribiendo código: requiere planificación, diseño y mantenimiento continuo para que los scripts sigan siendo útiles a medida que el software evoluciona.



Cada fase del flujo agrega valor: la **planificación** evita trabajo innecesario, el **reporte** hace los resultados accionables y el **mantenimiento** garantiza que la suite de pruebas no se vuelva obsoleta.

Herramientas Populares de Automatización



Web UI

- **Selenium** — estándar de la industria, multi-lenguaje
- **Cypress** — JavaScript, ideal para apps modernas
- **Playwright** — Microsoft, multi-browser avanzado

Unit / Integración

- **JUnit** — estándar para Java
- **TestNG** — potente suite para Java empresarial
- **pytest** — popular en proyectos Python

APIs

- **Postman** — pruebas y documentación de APIs REST
- **REST Assured** — automatización de APIs en Java
- **k6** — pruebas de carga y rendimiento

Validación Funcional de Aplicaciones Web

¿Qué se valida?

- **Interfaz de usuario (UI)** — elementos visibles y navegación
- **Flujos de negocio** — login, compra, registro
- **Formularios** — validaciones, mensajes de error
- **APIs** — respuestas, códigos HTTP, payloads
- **Rendimiento** — tiempos de respuesta aceptables

Ejemplo: Caso de Prueba de Login

```
// Caso de prueba: Login exitoso
// Precondición: usuario registrado

DADO que el usuario está en /login
CUANDO ingresa email: "user@test.com"
Y contraseña: "Pass1234!"
Y hace clic en "Iniciar sesión"
ENTONCES debe redirigir a /dashboard
Y mostrar "Bienvenido, Usuario"
Y el token JWT debe existir
```


Ejemplo Real: Prueba Automatizada con Cypress

El siguiente snippet muestra un test funcional completo para validar el flujo de login. Cada línea está comentada para facilitar su comprensión.

```
// test/login.spec.cy.js – Cypress Test Suite

describe('Flujo de Login', () => {

  beforeEach(() => {
    cy.visit('/login')          // 1. Navegar a la página de login
  })

  it('Login exitoso con credenciales válidas', () => {
    cy.get('#email')            // 2. Seleccionar campo email
      .type('user@demo.com')    //   Ingresar email de prueba

    cy.get('#password')         // 3. Seleccionar campo contraseña
      .type('SecurePass123!')   //   Ingresar contraseña

    cy.get('[data-cy=btn-login]') // 4. Buscar botón por atributo data
      .click()                  //   Simular clic

    cy.url()                    // 5. Verificar URL resultante
      .should('include', '/dashboard')

    cy.contains('Bienvenido')   // 6. Confirmar mensaje de éxito
      .should('be.visible')
  })
})
```

 La convención `data-cy` es una buena práctica: evita que los tests se rompan si cambia el CSS o la estructura HTML.

IA Aplicada al QA y Testing con Agentes

La inteligencia artificial está transformando radicalmente cómo se diseñan, ejecutan y mantienen las pruebas de software. Los **agentes autónomos de QA** pueden explorar una aplicación, detectar anomalías y generar casos de prueba sin intervención humana.



Generación Automática de Tests

GitHub Copilot sugiere código de prueba basándose en el contexto del código fuente, reduciendo el tiempo de escritura hasta un 40%.



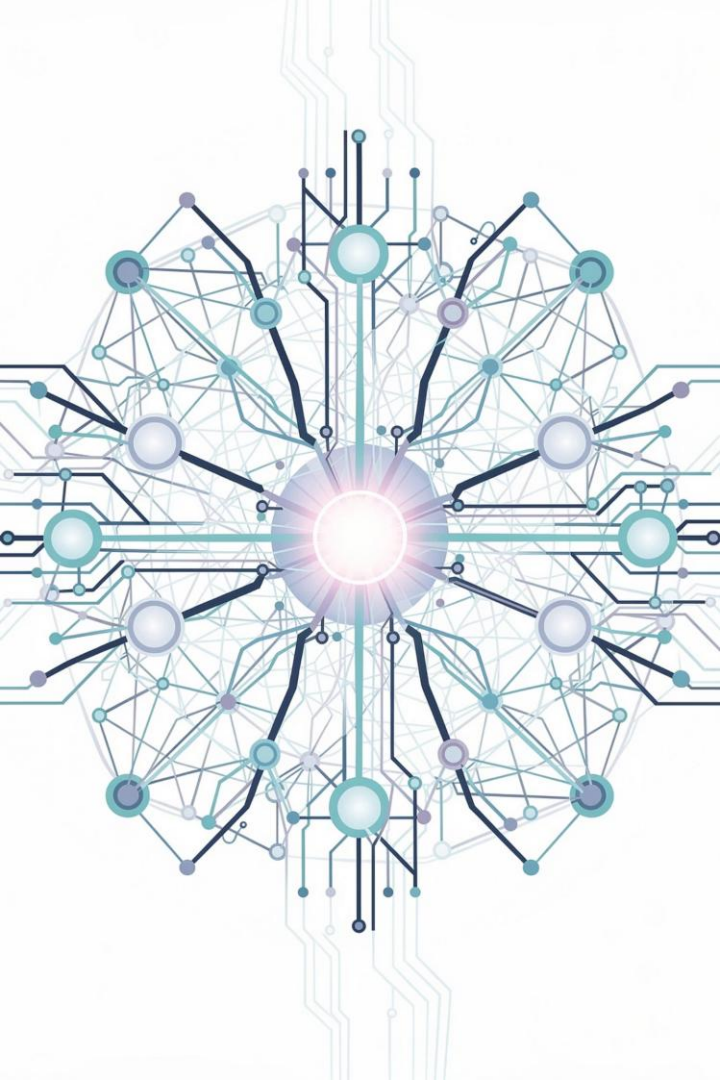
Análisis Inteligente de Logs

Mabl analiza logs de ejecución con ML para identificar patrones de fallos y priorizar qué bugs corregir primero.



Agentes Autónomos de QA

Testim usa IA para auto-reparar selectores rotos cuando la UI cambia, eliminando el mantenimiento manual de scripts.



¿Por Qué Importa la Seguridad en el Software?

Las brechas de seguridad no son hipotéticas — son costosas, frecuentes y devastadoras para empresas y usuarios.

\$4.45M

Costo promedio de una brecha

Costo global promedio de una violación de datos en 2023, según IBM Security.

83%

Organizaciones afectadas

De las organizaciones ha sufrido más de una brecha de datos en su historia.

277

Días promedio

Tiempo promedio para identificar y contener una brecha de seguridad en 2023.

60%

PYMES cierran

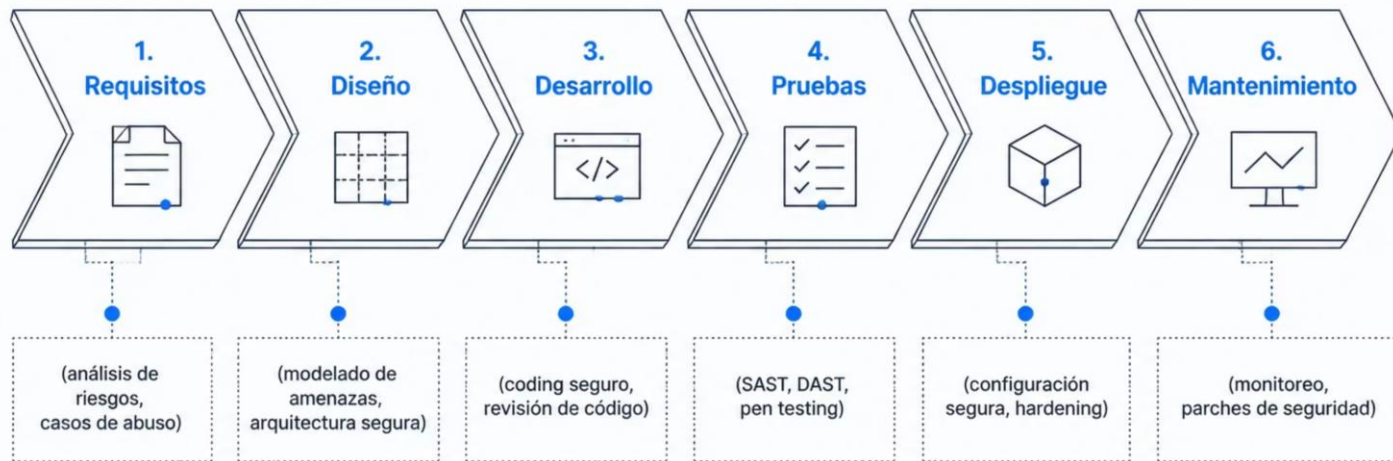
De las pequeñas empresas cierra dentro de los 6 meses posteriores a un ciberataque.

«La seguridad no es una característica opcional — es un requisito no funcional fundamental desde el primer día de desarrollo.»



Seguridad en el Ciclo de Vida del Software (SDLC)

La seguridad debe integrarse en **cada fase** del desarrollo, no añadirse al final como un parche. Este enfoque se conoce como **DevSecOps**.



Shift Left Security: Mover la Seguridad hacia la Izquierda

Shift Left Security es el principio de anticipar las actividades de seguridad lo más temprano posible en el ciclo de desarrollo. El nombre viene de visualizar el SDLC como una línea horizontal: "mover a la izquierda" significa actuar antes.

La lógica económica es contundente: cuanto más tarde se detecta una vulnerabilidad, exponencialmente más costosa resulta su corrección.



Detectado en Diseño

Costo relativo: **1x** — solo requiere cambiar un diagrama.



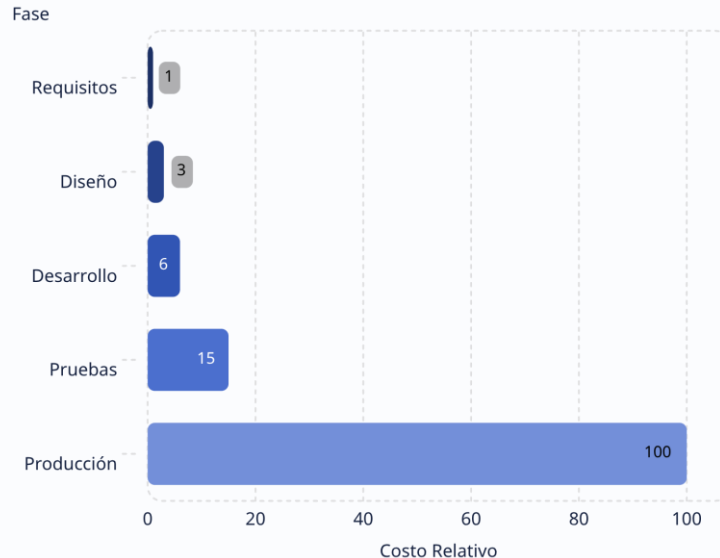
Detectado en Desarrollo

Costo relativo: **6x** — hay que refactorizar código ya escrito.



Detectado en Producción

Costo relativo: **100x** — impacto en usuarios reales, parches urgentes y reputación.



Buenas Prácticas de Desarrollo Seguro



Validación de Entradas

Nunca confiar en datos del usuario.
Sanitizar y validar **todo** input antes de procesarlo o almacenarlo.



Principio de Mínimo Privilegio

Cada componente y usuario debe tener **solo los permisos estrictamente necesarios** para su función.



Cifrado de Datos

Datos sensibles deben cifrarse en tránsito (TLS/HTTPS) y en reposo (AES-256) sin excepción.



Manejo Seguro de Errores

Los mensajes de error **no deben revelar** información interna del sistema (stack traces, rutas, versiones).

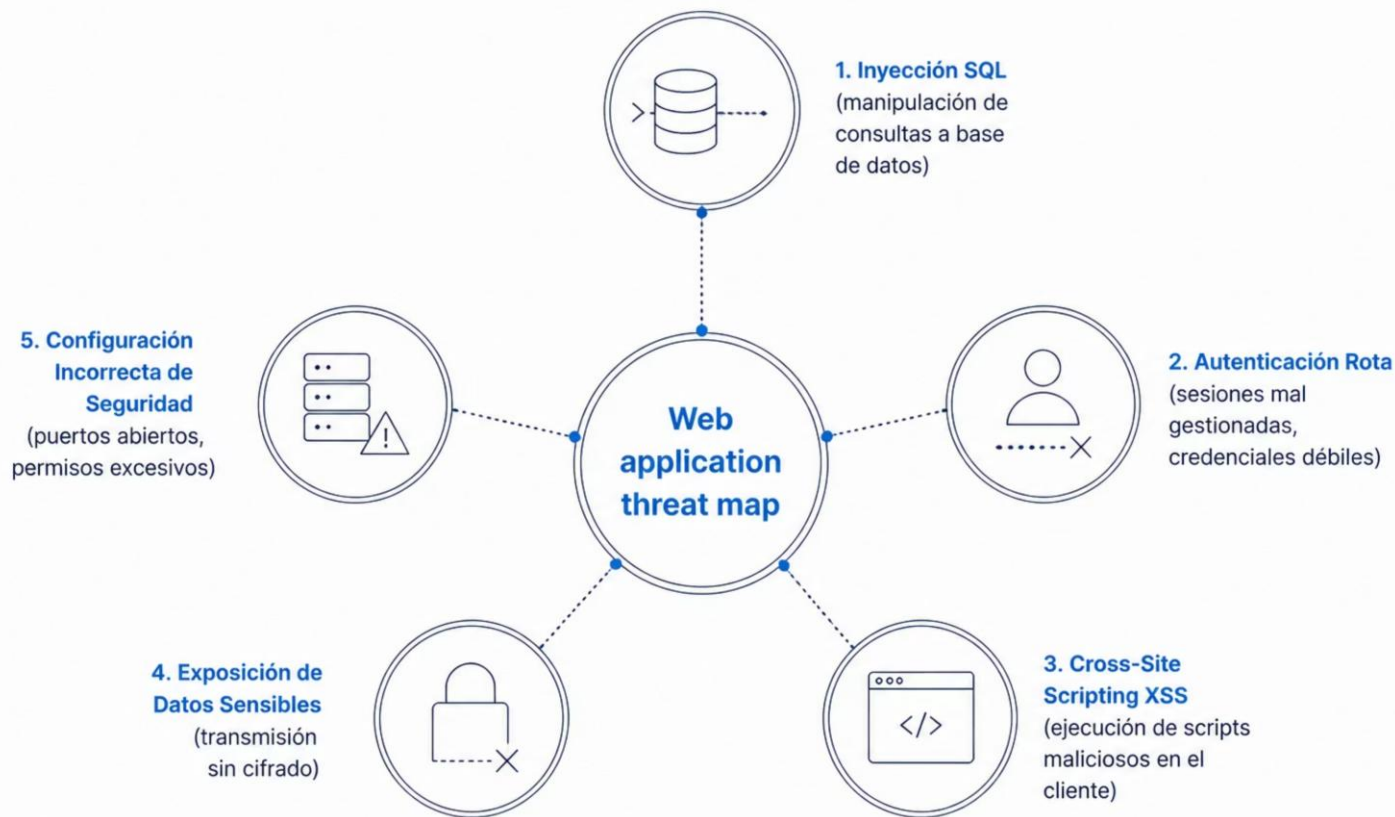


Revisión de Código (Code Review)

Revisiones por pares con checklist de seguridad detectan vulnerabilidades antes de que lleguen a producción.

Seguridad en Aplicaciones Web

Las aplicaciones web son el vector de ataque más explotado globalmente. Conocer las amenazas principales es el primer paso para defenderse.



Seguridad en Aplicaciones Móviles

Riesgos Específicos en Mobile

Las aplicaciones móviles presentan una superficie de ataque diferente a las web. Los principales vectores incluyen:

Almacenamiento Inseguro

Guardar tokens, contraseñas o datos sensibles en almacenamiento local sin cifrar (SharedPreferences, SQLite sin encrypt).

Comunicaciones No Cifradas

Usar HTTP en lugar de HTTPS, o no validar certificados SSL, permite ataques de tipo **Man-in-the-Middle**.

Permisos Excesivos

Solicitar acceso a cámara, micrófono, contactos o ubicación sin necesidad real expone datos del usuario innecesariamente.



OWASP Top 10 – Vulnerabilidades #1 al #5

El **OWASP Top 10** es el estándar de referencia global para la seguridad de aplicaciones web. Conocerlo es obligatorio para cualquier desarrollador profesional.

1

Broken Access Control

Usuarios acceden a recursos o funciones para los que no tienen permiso. El más explotado en 2021-2023.

2

Cryptographic Failures

Datos sensibles transmitidos o almacenados sin cifrado adecuado. Antes llamado "Exposición de datos sensibles".

3

Injection

SQL, NoSQL, OS o LDAP injection. El atacante envía datos hostiles a un intérprete como parte de un comando.

4

Insecure Design

Fallas en el diseño arquitectónico que ningún control de implementación puede corregir completamente.

5

Security Misconfiguration

Configuraciones incorrectas: puertos abiertos innecesarios, cuentas por defecto activas, permisos excesivos en cloud.

OWASP Top 10 — Vulnerabilidades #6 al #10 y Mitigaciones

#	Vulnerabilidad	Descripción	Mitigación Principal
6	Vulnerable & Outdated Components	Uso de librerías, frameworks o SO con vulnerabilidades conocidas sin parchear.	Mantener inventario de dependencias y actualizar regularmente. Usar herramientas como Dependabot o Snyk.
7	Identification & Auth Failures	Sesiones mal gestionadas, contraseñas débiles, falta de MFA en funciones críticas.	Implementar MFA, limitar intentos de login, usar gestores de sesión probados.
8	Software & Data Integrity Failures	Deserialización insegura o actualizaciones automáticas sin verificación de firma digital.	Verificar firmas digitales e integridad de paquetes. Revisar pipelines CI/CD.
9	Security Logging & Monitoring Failures	Sin logs adecuados, los ataques no se detectan hasta que el daño ya está hecho.	Implementar logging centralizado, alertas automáticas y planes de respuesta a incidentes.
10	Server-Side Request Forgery (SSRF)	El servidor realiza solicitudes a destinos controlados por el atacante, exponiendo recursos internos.	Validar y sanitizar todas las URLs generadas por el usuario. Usar listas blancas de dominios permitidos.

Resumen y Próximos Pasos

● Bloque 1: Testing Automatizado

- La automatización **complementa** las pruebas manuales — no las reemplaza.
- El flujo: Planificación → Script → Ejecución → Reporte → Mantenimiento.
- Herramientas clave: **Cypress, Selenium, Postman, JUnit.**
- La IA (Copilot, Testim, Mabl) está transformando el QA con agentes autónomos.

● Bloque 2: Seguridad en el Software

- **Shift Left:** integrar seguridad desde el primer día reduce costos hasta 100x.
- Buenas prácticas: validar entradas, cifrar datos, mínimo privilegio.
- Web y Mobile tienen superficies de ataque distintas — ambas deben cubrirse.
- **OWASP Top 10** es la referencia obligatoria para todo desarrollador.